Generative Artificial Intelligence

Lab 1: Autoencoders and Variational Autoencoders

with PyTorch, on the MNIST dataset

Objectives

At the end of this lab, you will be able to:

- prepare a Python environment for deep learning (Anaconda, PyTorch) and check if a GPU is available;
- load the MNIST dataset with `torchvision` and use a `DataLoader`;
- build and train a **linear autoencoder**, and understand what the *code* (compressed representation) is;
- build a **convolutional autoencoder**, with `Conv2d` in the encoder and `ConvTranspose2d` in the decoder, and check the size of every layer with the formulas of the lecture;
- train a **denoising autoencoder** (noisy input, clean target) and measure how much noise it removes;
- build and train a **variational autoencoder (VAE)**: two-headed encoder, reparameterization trick, loss = reconstruction + KL;
- use a trained VAE to **generate** new digits, to interpolate between two digits, and to visualize a 2-D latent space;
- explain why a normal autoencoder cannot generate new images, and why a VAE can.

This document explains every block of code of the lab. Read the explanation, run the code, look at the output, then answer the questions. The Python packages needed for the lab are listed in the file `requirements.txt`.

Section 0 — Prepare the environment

0.1 Create a new conda environment by cloning `base`

We do not install packages directly in the `base` environment of Anaconda: if something breaks, the whole installation is broken. We make a **copy** of `base` and we work in the copy. The copy already contains NumPy, Matplotlib and Jupyter.

Open **Anaconda Prompt** (Windows) or a terminal (Linux, macOS) and type:

```
conda create --name genai --clone base
conda activate genai
```

- `conda create --name genai --clone base`: creates an environment named `genai` that is a copy of `base`. It takes a few minutes and several GB of disk space.
- `conda activate genai`: enters the new environment. The prompt now starts with `(genai)`. **You must run this command every time you open a new terminal.**
- `conda env list`: shows all your environments (the active one has a star).

0.2 Install PyTorch and the other packages

PyTorch is not part of Anaconda: we must install it. There are two cases.

Case 1: no NVIDIA GPU (or you do not know). Install the default version:

```
pip install torch torchvision
```

Case 2: you have an NVIDIA GPU. First check the driver: the command `nvidia-smi` must print a table with the name of your GPU. Then open https://pytorch.org/get-started/locally/, choose *Stable*, your operating system, *Pip*, *Python* and a CUDA version, and copy the command that the page gives. It has this form:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cuXXX
```

where `cuXXX` is the CUDA version chosen on the page (for example `cu126` for CUDA 12.6). Always copy the command from the official page: the available CUDA versions change with every PyTorch release.

Important
On Windows, the simple command of Case 1 installs a version of PyTorch that works **only on the CPU**, even if you have a GPU. To use the GPU, you need the command of Case 2.

Then install the other packages of the lab and make the environment visible in Jupyter:

```
pip install -r requirements.txt
python -m ipykernel install --user --name genai --display-name "Python (genai)"
jupyter notebook
```

- `pip install -r requirements.txt`: installs every package listed in the file. Packages that are already installed with a good version are not changed (so a GPU version of PyTorch installed in Case 2 is kept).
- `python -m ipykernel install ...`: adds a kernel named "Python (genai)" to Jupyter. Choose it in the menu *Kernel* → *Change kernel*.
- `jupyter notebook`: opens Jupyter in your browser. Run it from the folder of the lab.

0.3 Verify the installation: is CUDA available?

Run the following blocks of code one by one (in Jupyter: **Shift + Enter**).

```
import sys, platform
import numpy as np
import matplotlib
import torch
import torchvision

print("Python      :", sys.version.split()[0], "on", platform.system())
print("PyTorch     :", torch.__version__)
print("torchvision :", torchvision.__version__)
print("NumPy       :", np.__version__)
print("Matplotlib  :", matplotlib.__version__)
```

If an import fails, the package is not installed *in the environment that runs your code*: check that the kernel is "Python (genai)".

```
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
```

```
    print("Number of GPUs:", torch.cuda.device_count())
    print("GPU name      :", torch.cuda.get_device_name(0))
    print("CUDA version  :", torch.version.cuda)
else:
    print("No GPU found: the lab will run on the CPU (slower, but it works).")
```

`torch.cuda.is_available()` answers the question: *can PyTorch use an NVIDIA GPU on this machine?*

- **`True`**: the GPU will be used. Training is much faster.
- **`False`**: there is no NVIDIA GPU, or the driver is missing, or you installed the CPU-only version of PyTorch (see the note above). **The lab still works on the CPU**, it is only slower.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("We will use:", device)

x = torch.rand(3, 3).to(device)    # create a tensor and move it to the device
y = x @ x                          # a matrix product, computed on that device
print(y)
print("The result lives on:", y.device)
```

This is the most important habit of the lab. We never write `"cuda"` directly in our code. We create `device` once, then we move the model and every batch of data to it with `.to(device)`. The same code runs on a GPU and on a CPU. A model and its data must always be on the **same** device, otherwise PyTorch raises an error.

The first time you load MNIST (Section 1.2), the dataset is downloaded once (about 12 MB) into a folder named `data`.

0.4 No GPU? Use Google Colab or Kaggle

Both services give a free GPU in the browser. PyTorch is already installed there: you can skip 0.1 and 0.2.

Google Colab (https://colab.research.google.com)

1. Sign in with a Google account.
2. *File* → *New notebook*.
3. *Runtime* → *Change runtime type*, choose a **GPU** (T4), then *Save*.
4. Type or paste the code of the lab and run it.

Kaggle (https://www.kaggle.com/code)

1. Sign in and verify your phone number (needed for the GPU and for Internet access).
2. *Create* → *New Notebook*.
3. In the right panel (*Settings*): *Accelerator* → **GPU**, and *Internet* → **On**.
4. Type or paste the code of the lab and run it.

In both cases, run the code of 0.3 first: it must print `CUDA available: True`. The sessions are temporary: download your work when you finish. The GPU time is limited (a quota per session on Colab, per week on Kaggle).

Section 1 — A simple (linear) autoencoder

Goal. Compress each MNIST digit (784 pixels) into a **code** of 32 numbers, and rebuild the image from the code:

$$\text{image (784)} \xrightarrow{\text{encoder}} \text{code (32)} \xrightarrow{\text{decoder}} \text{reconstruction (784)}$$

1.1 Imports and device

```
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms

%matplotlib inline

torch.manual_seed(0)      # same random numbers at every run (reproducible results)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("PyTorch version:", torch.__version__)
print("Device:", device)
```

- `torch.nn` (as `nn`) contains the layers; `torch.nn.functional` (as `F`) contains functions such as `relu`.
- `torchvision` gives the MNIST dataset and the image transformations.
- `%matplotlib inline` shows the figures inside Jupyter.
- `torch.manual_seed(0)` fixes the random numbers: you get the same results at every run.

1.2 Load the data

```
# ToTensor() turns a PIL image into a tensor of shape (1, 28, 28) with values in [0, 1]
transform = transforms.ToTensor()

# download=True: the files are downloaded once into the folder "data", then reused
train_data = datasets.MNIST(root="data", train=True, download=True, transform=transform)
test_data = datasets.MNIST(root="data", train=False, download=True, transform=transform)

batch_size = 64
train_loader = torch.utils.data.DataLoader(train_data, batch_size=batch_size, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_data, batch_size=batch_size, shuffle=False)

print("Training images:", len(train_data), "| Test images:", len(test_data))
images, labels = next(iter(train_loader))      # one batch
print("One batch of images:", tuple(images.shape))
print("Pixel values between", images.min().item(), "and", images.max().item())
```

- A `Dataset` gives the images one by one. A `DataLoader` groups them in **batches** of 64 images. `shuffle=True` mixes the training images at every epoch.
- One batch has the shape `(64, 1, 28, 28)`: 64 images, 1 channel (gray), 28×28 pixels.
- `next(iter(train_loader))` takes one batch. (Old tutorials write `dataiter.next()`: this no longer works.)

1.3 Look at the data

```
def show_rows(rows, titles, n=10):
    # Display several rows of n images. Each element of "rows" is a tensor
    # with n (or more) images, of shape (N, 1, 28, 28) or (N, 784).
    fig, axes = plt.subplots(len(rows), n, figsize=(1.3 * n, 1.55 * len(rows)),
                             gridspec_kw={"wspace": 0.05, "hspace": 0.3})
    axes = np.atleast_2d(axes)
    for r, (imgs, title) in enumerate(zip(rows, titles)):
        imgs = imgs.detach().cpu().numpy().reshape(-1, 28, 28)
        for i in range(n):
```

```
            axes[r, i].imshow(imgs[i], cmap="gray", vmin=0, vmax=1)
            axes[r, i].axis("off")
        axes[r, 0].set_title(title, fontsize=11, loc="left")
    plt.show()

show_rows([images], ["Ten training images"])
print("Labels:", labels[:10].tolist())
```

`show_rows` draws several rows of images. We reuse it in the whole lab. `.detach().cpu().numpy()` converts a tensor (maybe on the GPU, maybe with gradients) into a NumPy array that Matplotlib can draw.

1.4 The model

```
class Autoencoder(nn.Module):
    def __init__(self, encoding_dim):
        super().__init__()
        # encoder: one linear layer, 784 -> encoding_dim
        self.fc1 = nn.Linear(28 * 28, encoding_dim)
        # decoder: one linear layer, encoding_dim -> 784
        self.fc2 = nn.Linear(encoding_dim, 28 * 28)

    def encode(self, x):
        return F.relu(self.fc1(x))           # the code (compressed representation)

    def decode(self, z):
        return torch.sigmoid(self.fc2(z))    # sigmoid: outputs in [0, 1], like pixels

    def forward(self, x):
        return self.decode(self.encode(x))

encoding_dim = 32
model = Autoencoder(encoding_dim).to(device)   # .to(device): put the weights on the GPU if any
print(model)
print("Number of parameters:", sum(p.numel() for p in model.parameters()))
```

- A PyTorch model is a class that inherits from `nn.Module`. The layers are created in `__init__`; the computation is written in `forward`.
- **Encoder**: `fc1`, a linear layer $784 \to 32$, then ReLU. **Decoder**: `fc2`, a linear layer $32 \to 784$, then a **sigmoid**, because the pixels are between 0 and 1.
- We separate `encode` and `decode` to be able to use each half alone (1.9 and 1.10).
- Parameters: $(784 \times 32 + 32) + (32 \times 784 + 784) = 50\,992$.

1.5 Loss and optimizer

```
criterion = nn.MSELoss()                                  # mean squared error
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)   # updates the weights
```

We compare pixel values, so we use a regression loss: the mean squared error. Adam updates the weights.

1.6 Training

```
n_epochs = 10       # increase it if you have a GPU
train_losses, test_losses = [], []
```

```
for epoch in range(1, n_epochs + 1):
    # ---------------- training ----------------
    model.train()
    running_loss = 0.0
    for images, _ in train_loader:                  # "_" = the labels: we do not use them
        images = images.to(device)
        x = images.view(images.size(0), -1)         # flatten: (B, 1, 28, 28) -> (B, 784)
        optimizer.zero_grad()                       # 1. reset the gradients
        outputs = model(x)                          # 2. forward pass
        loss = criterion(outputs, x)                # 3. compare the output with the INPUT
        loss.backward()                             # 4. backward pass (compute gradients)
        optimizer.step()                            # 5. update the weights
        running_loss += loss.item() * images.size(0)
    train_loss = running_loss / len(train_loader.dataset)    # average over all images

    # ---------------- evaluation on the test set ----------------
    model.eval()
    running_loss = 0.0
    with torch.no_grad():                           # no gradients needed here
        for images, _ in test_loader:
            x = images.to(device).view(images.size(0), -1)
            running_loss += criterion(model(x), x).item() * images.size(0)
    test_loss = running_loss / len(test_loader.dataset)

    train_losses.append(train_loss)
    test_losses.append(test_loss)
    print(f"Epoch {epoch:2d}/{n_epochs} | train loss: {train_loss:.5f} | test loss: {test_loss:.5f}"
    )
```

- `for images, _ in train_loader`: the labels are ignored. An autoencoder is **unsupervised**.
- `images.view(images.size(0), -1)`: a linear layer needs vectors, so each image is flattened to 784 values.
- The five steps are the same as for any network. Only step 3 is special: the loss compares the output with the **input** (not with a label).
- `loss.item()` is the mean over the batch. We multiply by the batch size and divide by the number of images at the end: this gives the true average per image.
- `model.eval()` and `torch.no_grad()`: on the test set we only measure, we do not learn.

Expected output (the values can change a little):

```
Epoch  1/10 | train loss: 0.04618 | test loss: 0.02673
Epoch  2/10 | train loss: 0.02262 | test loss: 0.01928
...
Epoch  9/10 | train loss: 0.01458 | test loss: 0.01414
Epoch 10/10 | train loss: 0.01451 | test loss: 0.01410
```

1.7 to 1.9 Learning curves, reconstructions, codes

```
plt.figure(figsize=(6, 3.5))
plt.plot(range(1, n_epochs + 1), train_losses, "o-", label="train")
plt.plot(range(1, n_epochs + 1), test_losses, "s-", label="test")
plt.xlabel("epoch")
plt.ylabel("MSE loss")
plt.title("Linear autoencoder: learning curves")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

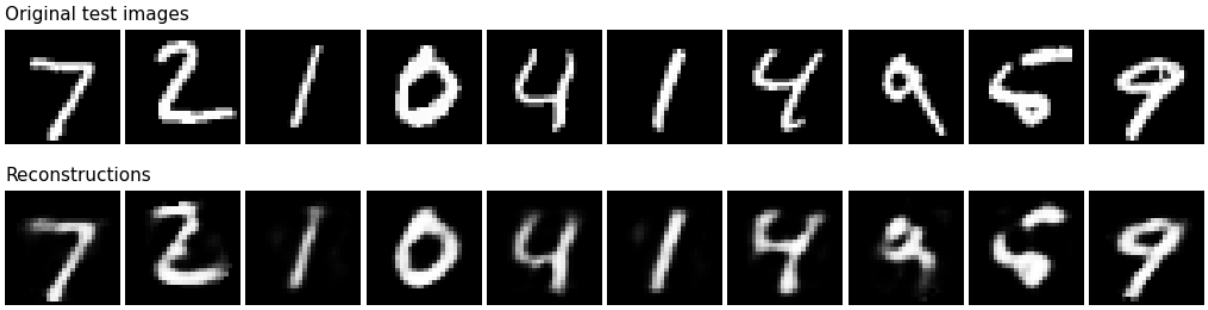
```
model.eval()
images, labels = next(iter(test_loader))          # one batch of TEST images
```

```
x = images.to(device).view(images.size(0), -1)
with torch.no_grad():
    codes = model.encode(x)                          # (B, 32)
    outputs = model.decode(codes)                    # (B, 784)

show_rows([images, outputs], ["Original test images", "Reconstructions"])
```

```
print("Shape of the codes:", tuple(codes.shape))
print(f"Compression: 784 values -> {encoding_dim} values ({784 / encoding_dim:.1f} times smaller)")

# show each 32-value code as a small 4 x 8 image
fig, axes = plt.subplots(1, 10, figsize=(14, 1.3))
for i, ax in enumerate(axes):
    ax.imshow(codes[i].cpu().numpy().reshape(4, 8), cmap="gray")
    ax.axis("off")
axes[0].set_title("Codes of the ten images above", fontsize=11, loc="left")
plt.show()
```

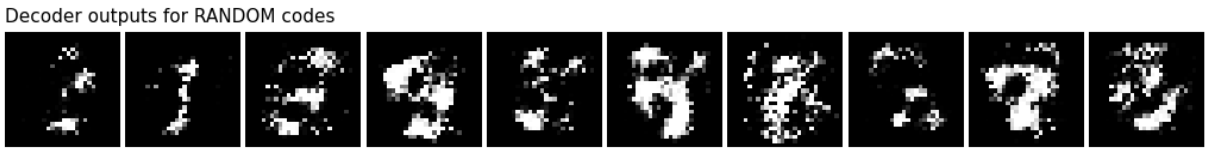


The reconstructions are a little blurry: some details are lost in the compression. The codes look like noise to us, but they contain enough information to rebuild the digits.

1.10 Can this autoencoder generate new digits?

```
# Try to GENERATE digits: give random codes to the decoder.
# We use random codes with the same range of values as the real codes.
torch.manual_seed(1)
random_codes = torch.rand(10, encoding_dim, device=device) * codes.max()
with torch.no_grad():
    generated = model.decode(random_codes)

show_rows([generated], ["Decoder outputs for RANDOM codes"])
```



The outputs are **not** digits. The autoencoder was only trained to reconstruct. Nothing organizes the space of the codes, so a random code falls in a region that the decoder has never seen. This is why we need the VAE (Section 4).

1.11 Experiment: the size of the code

```python
def train_linear_ae(encoding_dim, n_epochs=3):
    # train a new autoencoder and return it with its final test loss
    torch.manual_seed(0)
    net = Autoencoder(encoding_dim).to(device)
    opt = torch.optim.Adam(net.parameters(), lr=0.001)
    for epoch in range(n_epochs):
        net.train()
        for images, _ in train_loader:
            x = images.to(device).view(images.size(0), -1)
            opt.zero_grad()
            loss = criterion(net(x), x)
            loss.backward()
            opt.step()
    net.eval()
    total = 0.0
    with torch.no_grad():
        for images, _ in test_loader:
            x = images.to(device).view(images.size(0), -1)
            total += criterion(net(x), x).item() * images.size(0)
    return net, total / len(test_loader.dataset)

dims = [2, 8, 32, 128]
results = {}
for d in dims:
    results[d] = train_linear_ae(d)
    print(f"encoding_dim = {d:3d} | test loss after 3 epochs: {results[d][1]:.5f}")
```

```python
images, _ = next(iter(test_loader))
x = images.to(device).view(images.size(0), -1)
rows, titles = [images], ["Original test images"]
with torch.no_grad():
    for d in dims:
        rows.append(results[d][0](x))
        titles.append(f"encoding_dim = {d}  (test loss {results[d][1]:.4f})")
show_rows(rows, titles)
```

We train four autoencoders with codes of size 2, 8, 32 and 128, three epochs each, and we compare their reconstructions.

Question 1.1. Compute the number of parameters by hand and compare with the printed value.

Question 1.2. Why do we use a sigmoid at the output? What happens if you replace it with a ReLU?

Question 1.3. Describe the effect of `encoding_dim` on the reconstructions. What would happen with `encoding_dim = 784`?

Question 1.4. (Exercise) Build a *deep* autoencoder $784 \to 128 \to 64 \to 32 \to 64 \to 128 \to 784$ (ReLU between the layers, sigmoid at the end). Train it for the same number of epochs and compare its test loss with the simple model.

Section 2 — A convolutional autoencoder

Goal. A linear layer forgets that an image is a 2-D grid. We now use **convolutions**: the encoder makes the image smaller, the decoder makes it bigger again.

$$(1, 28, 28) \to (16, 14, 14) \to \underbrace{(4, 7, 7)}_{\text{code}} \to (16, 14, 14) \to (1, 28, 28)$$

The imports and the data (1.1 and 1.2) are the same as in Section 1.

2.3 The model

```
class ConvAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        # ---------- encoder ----------
        self.conv1 = nn.Conv2d(1, 16, 3, padding=1)    # depth 1 -> 16, size unchanged
        self.conv2 = nn.Conv2d(16, 4, 3, padding=1)    # depth 16 -> 4, size unchanged
        self.pool = nn.MaxPool2d(2, 2)                 # divides height and width by 2
        # ---------- decoder ----------
        # kernel 2 and stride 2: multiplies height and width by 2
        self.t_conv1 = nn.ConvTranspose2d(4, 16, 2, stride=2)
        self.t_conv2 = nn.ConvTranspose2d(16, 1, 2, stride=2)

    def encode(self, x):
        x = self.pool(F.relu(self.conv1(x)))           # (B, 16, 14, 14)
        x = self.pool(F.relu(self.conv2(x)))           # (B, 4, 7, 7)  <- the code
        return x

    def decode(self, z):
        x = F.relu(self.t_conv1(z))                    # (B, 16, 14, 14)
        x = torch.sigmoid(self.t_conv2(x))             # (B, 1, 28, 28), values in [0, 1]
        return x

    def forward(self, x):
        return self.decode(self.encode(x))

model = ConvAutoencoder().to(device)
print(model)
print("Number of parameters:", sum(p.numel() for p in model.parameters()))
```

- `nn.Conv2d(1, 16, 3, padding=1)`: 1 input channel, 16 output channels, a 3×3 kernel. With `padding=1` the height and width do not change.
- `nn.MaxPool2d(2, 2)`: keeps the maximum of each 2×2 block. Height and width are divided by 2.
- `nn.ConvTranspose2d(4, 16, 2, stride=2)`: a transposed convolution with kernel 2 and stride 2. It multiplies the height and width by 2. Formula of the lecture:

$$\text{out} = (\text{in} - 1) \times \text{stride} - 2 \times \text{padding} + \text{kernel} = (7 - 1) \times 2 - 0 + 2 = 14$$

- The model has only 1 077 parameters (the linear autoencoder has 50 992): a convolution reuses the same small kernel at every position.

2.4 Follow the shapes, layer by layer

```
# Follow one batch through the network, layer by layer
x = images[:1].to(device)                  # one image
print("input              :", tuple(x.shape))
with torch.no_grad():
    x = F.relu(model.conv1(x)); print("after conv1        :", tuple(x.shape))
    x = model.pool(x);          print("after pool         :", tuple(x.shape))
    x = F.relu(model.conv2(x)); print("after conv2        :", tuple(x.shape))
    x = model.pool(x);          print("after pool (code)  :", tuple(x.shape))
    x = F.relu(model.t_conv1(x)); print("after t_conv1      :", tuple(x.shape))
    x = torch.sigmoid(model.t_conv2(x)); print("after t_conv2      :", tuple(x.shape))

print("\nSize of the code: 4 x 7 x 7 =", 4 * 7 * 7, "values (the image has 784)")
```

Printing the shape after each layer is the best way to debug a convolutional network. Expected output:

```
input              : (1, 1, 28, 28)
after conv1        : (1, 16, 28, 28)
after pool         : (1, 16, 14, 14)
after conv2        : (1, 4, 14, 14)
after pool (code)  : (1, 4, 7, 7)
after t_conv1      : (1, 16, 14, 14)
after t_conv2      : (1, 1, 28, 28)

Size of the code: 4 x 7 x 7 = 196 values (the image has 784)
```

2.5 Training

```python
criterion = nn.MSELoss()

def evaluate(model):
    # average MSE on the test set
    model.eval()
    total = 0.0
    with torch.no_grad():
        for images, _ in test_loader:
            images = images.to(device)
            total += criterion(model(images), images).item() * images.size(0)
    return total / len(test_loader.dataset)

def train(model, n_epochs, lr=0.001):
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    history = {"train": [], "test": []}
    for epoch in range(1, n_epochs + 1):
        model.train()
        running_loss = 0.0
        for images, _ in train_loader:
            images = images.to(device)             # NO flattening: we keep (B, 1, 28, 28)
            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, images)      # the target is the input image
            loss.backward()
            optimizer.step()
            running_loss += loss.item() * images.size(0)
        history["train"].append(running_loss / len(train_loader.dataset))
        history["test"].append(evaluate(model))
        print(f"Epoch {epoch:2d}/{n_epochs} | train loss: {history['train'][-1]:.5f}"
              f" | test loss: {history['test'][-1]:.5f}")
    return history
```

```python
n_epochs = 10       # increase it if you have a GPU
history = train(model, n_epochs)
```

The loop is the same as in Section 1, with one difference: we do **not** flatten the images, because a convolution needs the shape `(B, 1, 28, 28)`. We put the loop in a function `train`, because we will train two models.

2.6 and 2.7 Reconstructions and feature maps of the code

```python
model.eval()
images, labels = next(iter(test_loader))
with torch.no_grad():
    outputs = model(images.to(device))

show_rows([images, outputs], ["Original test images", "Reconstructions (transposed convolutions)"])
```

```python
# The code of one image is made of 4 small feature maps of size 7 x 7
```

```
with torch.no_grad():
    code = model.encode(images[:5].to(device)).cpu()     # (5, 4, 7, 7)

fig, axes = plt.subplots(5, 5, figsize=(6, 6))
for i in range(5):
    axes[i, 0].imshow(images[i, 0], cmap="gray")
    axes[i, 0].axis("off")
    for c in range(4):
        axes[i, c + 1].imshow(code[i, c], cmap="viridis")
        axes[i, c + 1].axis("off")
axes[0, 0].set_title("image", fontsize=10)
for c in range(4):
    axes[0, c + 1].set_title(f"map {c + 1}", fontsize=10)
plt.tight_layout()
plt.show()
```

Here the code is not a vector: it is a stack of 4 small maps of size 7×7. The maps still look like the digit: convolutions keep the spatial structure.

2.8 and 2.9 Another decoder: upsampling + convolution

```
class ConvAutoencoderUpsample(nn.Module):
    def __init__(self):
        super().__init__()
        # ---------- encoder (the same as before) ----------
        self.conv1 = nn.Conv2d(1, 16, 3, padding=1)
        self.conv2 = nn.Conv2d(16, 4, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        # ---------- decoder: normal convolutions ----------
        self.conv4 = nn.Conv2d(4, 16, 3, padding=1)
        self.conv5 = nn.Conv2d(16, 1, 3, padding=1)

    def encode(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        return x

    def decode(self, z):
        # upsample (nearest neighbour: no weights), THEN a normal convolution
        x = F.interpolate(z, scale_factor=2, mode="nearest")     # 7 -> 14
        x = F.relu(self.conv4(x))
        x = F.interpolate(x, scale_factor=2, mode="nearest")     # 14 -> 28
        x = torch.sigmoid(self.conv5(x))
        return x

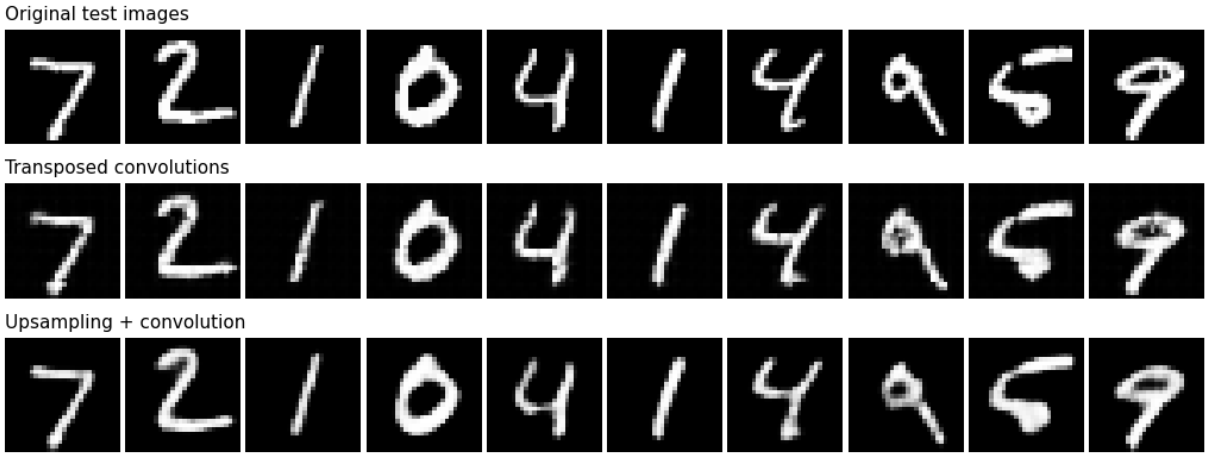
    def forward(self, x):
        return self.decode(self.encode(x))

torch.manual_seed(0)
model_up = ConvAutoencoderUpsample().to(device)
print("Number of parameters:", sum(p.numel() for p in model_up.parameters()))
history_up = train(model_up, n_epochs)
```

```
plt.figure(figsize=(6, 3.5))
plt.plot(range(1, n_epochs + 1), history["test"], "o-", label="transposed convolutions")
plt.plot(range(1, n_epochs + 1), history_up["test"], "s-", label="upsampling + convolution")
plt.xlabel("epoch")
plt.ylabel("test MSE")
plt.title("Two decoders, same encoder")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

```
with torch.no_grad():
    out_t = model(images.to(device))
    out_u = model_up(images.to(device))
show_rows([images, out_t, out_u],
          ["Original test images", "Transposed convolutions", "Upsampling + convolution"])
```

Transposed convolutions can create small *checkerboard* patterns. A common alternative is to make the image bigger with a fixed rule (`F.interpolate`, nearest neighbour, no weights) and then apply a normal convolution, which learns how to add the details.



Question 2.1. Why does this model have so few parameters compared with the linear autoencoder?

Question 2.2. The code has $4 \times 7 \times 7 = 196$ values. What is the compression ratio? Is the comparison with Section 1 (32 values) fair?

Question 2.3. Which decoder gives the lower test loss in your run? Look closely at the strokes: do you see a difference of texture between the two decoders?

Question 2.4. (Exercise) In the encoder, replace `MaxPool2d` by convolutions with `stride=2`. Check the shapes with the code of 2.4.

Section 3 — A denoising autoencoder

Goal. Train an autoencoder to remove noise. The **input** is a noisy image, the **target** is the clean image:

$$\text{clean image} \xrightarrow{+\,\text{noise}} \text{noisy image} \xrightarrow{\text{autoencoder}} \text{output} \quad \text{(compared with the } \textbf{clean} \text{ image)}$$

3.3 Add noise to the images

```
def add_noise(images, noise_factor=0.5):
    # add Gaussian noise, then keep the pixel values between 0 and 1
    noisy = images + noise_factor * torch.randn_like(images)
    return torch.clamp(noisy, 0.0, 1.0)

noise_factor = 0.5
noisy_images = add_noise(images, noise_factor)
show_rows([images, noisy_images], ["Clean images", f"Noisy images (noise_factor = {noise_factor})"])
```

- `torch.randn_like(images)`: a tensor of Gaussian noise $\mathcal{N}(0, 1)$ with the same shape and on the same device as `images`.

- `noise_factor` controls the amount of noise.
- `torch.clamp(noisy, 0.0, 1.0)`: values below 0 become 0, values above 1 become 1. The noisy image is still a valid image.

3.4 The model

```
class ConvDenoiser(nn.Module):
    def __init__(self):
        super().__init__()
        # ---------- encoder ----------
        self.conv1 = nn.Conv2d(1, 32, 3, padding=1)     # depth 1 -> 32
        self.conv2 = nn.Conv2d(32, 16, 3, padding=1)    # depth 32 -> 16
        self.conv3 = nn.Conv2d(16, 8, 3, padding=1)     # depth 16 -> 8
        self.pool = nn.MaxPool2d(2, 2)                  # size / 2
        # ---------- decoder ----------
        self.t_conv1 = nn.ConvTranspose2d(8, 8, 3, stride=2)     # kernel 3: 3 -> 7
        self.t_conv2 = nn.ConvTranspose2d(8, 16, 2, stride=2)    # kernel 2: 7 -> 14
        self.t_conv3 = nn.ConvTranspose2d(16, 32, 2, stride=2)   # kernel 2: 14 -> 28
        self.conv_out = nn.Conv2d(32, 1, 3, padding=1)           # depth 32 -> 1

    def forward(self, x):
        # encode
        x = self.pool(F.relu(self.conv1(x)))      # (B, 32, 14, 14)
        x = self.pool(F.relu(self.conv2(x)))      # (B, 16, 7, 7)
        x = self.pool(F.relu(self.conv3(x)))      # (B, 8, 3, 3)   <- the code
        # decode
        x = F.relu(self.t_conv1(x))               # (B, 8, 7, 7)
        x = F.relu(self.t_conv2(x))               # (B, 16, 14, 14)
        x = F.relu(self.t_conv3(x))               # (B, 32, 28, 28)
        x = torch.sigmoid(self.conv_out(x))       # (B, 1, 28, 28)
        return x

model = ConvDenoiser().to(device)
print(model)
print("Number of parameters:", sum(p.numel() for p in model.parameters()))
```

Denoising is harder than copying, so the network is **deeper** than in Section 2: three convolutions in the encoder (32, 16 and 8 channels) and three transposed convolutions in the decoder. The last layer is a normal convolution that reduces the depth from 32 to 1.

3.5 Check the sizes with the formulas

```
# Check the sizes with the formula of the lecture:
#   convolution / pooling  : out = floor((in + 2p - k) / s) + 1
#   transposed convolution : out = (in - 1) * s - 2p + k
print("pool on 7x7        :", (7 - 2) // 2 + 1)            # 3  (7 is odd: one row is lost)
print("t_conv1 (k=3, s=2) :", (3 - 1) * 2 + 3)             # 7
print("t_conv2 (k=2, s=2) :", (7 - 1) * 2 + 2)             # 14
print("t_conv3 (k=2, s=2) :", (14 - 1) * 2 + 2)            # 28

with torch.no_grad():
    out = model(images[:1].to(device))
print("input", tuple(images[:1].shape), "-> output", tuple(out.shape))
```

After the third pooling the maps are 3×3, not 3.5×3.5: 7 is odd, so one row and one column are lost. With a kernel of 2, the first transposed convolution would give $(3-1) \times 2 + 2 = 6$. To come back to 7 we need a kernel of **3**: $(3-1) \times 2 + 3 = 7$.

3.6 Training

```
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

n_epochs = 5        # increase it if you have a GPU
train_losses = []

for epoch in range(1, n_epochs + 1):
    model.train()
    running_loss = 0.0
    for images, _ in train_loader:
        images = images.to(device)
        noisy_images = add_noise(images, noise_factor)    # new noise at every batch
        optimizer.zero_grad()
        outputs = model(noisy_images)                     # the INPUT is the noisy image
        loss = criterion(outputs, images)                 # the TARGET is the clean image
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * images.size(0)
    train_losses.append(running_loss / len(train_loader.dataset))
    print(f"Epoch {epoch}/{n_epochs} | train loss: {train_losses[-1]:.5f}")
```

Only two lines differ from a normal autoencoder:

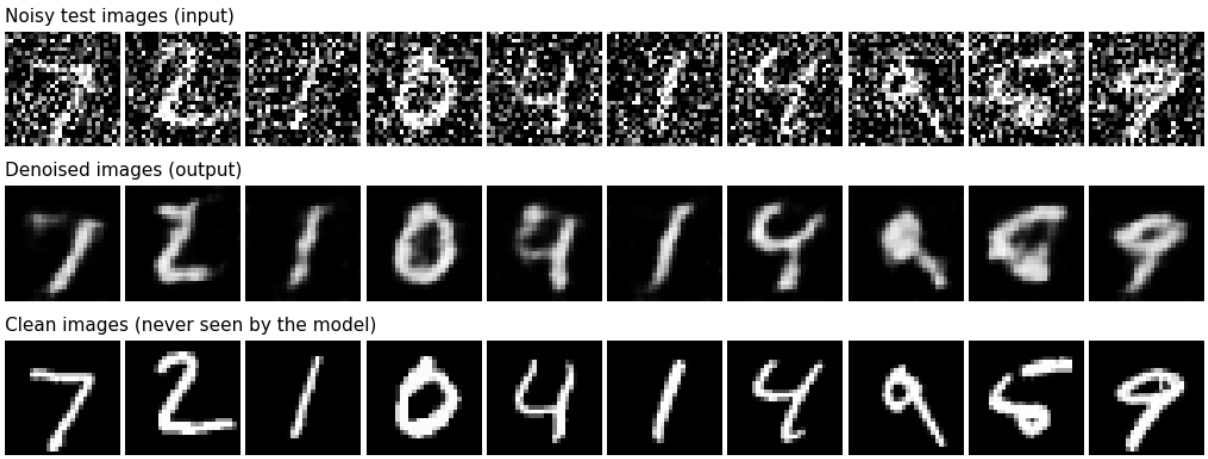
- `outputs = model(noisy_images)`: the input of the model is the **noisy** image;
- `loss = criterion(outputs, images)`: the target of the loss is the **clean** image.

The noise is new at every batch: the network never sees the same noisy image twice.

3.7 and 3.8 Results and measure of the improvement

```
model.eval()
torch.manual_seed(0)
images, labels = next(iter(test_loader))
noisy_images = add_noise(images, noise_factor)
with torch.no_grad():
    outputs = model(noisy_images.to(device))

show_rows([noisy_images, outputs, images],
          ["Noisy test images (input)", "Denoised images (output)", "Clean images (never seen by the
     model)"])
```




```
def test_errors(noise_factor):
    # MSE with the clean images: before denoising (noisy) and after (output)
    model.eval()
```

```
    before, after = 0.0, 0.0
    with torch.no_grad():
        for images, _ in test_loader:
            images = images.to(device)
            noisy = add_noise(images, noise_factor)
            before += criterion(noisy, images).item() * images.size(0)
            after += criterion(model(noisy), images).item() * images.size(0)
    n = len(test_loader.dataset)
    return before / n, after / n

before, after = test_errors(noise_factor)
print(f"MSE between noisy and clean images   : {before:.5f}")
print(f"MSE between denoised and clean images: {after:.5f}")
print(f"The error is divided by {before / after:.1f}")
```

We compare each image with the clean one, before denoising (the noisy input) and after (the output). Expected output:

```
MSE between noisy and clean images   : 0.11570
MSE between denoised and clean images: 0.02808
The error is divided by 4.1
```

3.9 Other noise levels

```
# The model was trained with noise_factor = 0.5. What about other noise levels?
levels = [0.0, 0.1, 0.3, 0.5, 0.7, 0.9]
errors = [test_errors(f) for f in levels]

plt.figure(figsize=(6, 3.5))
plt.plot(levels, [e[0] for e in errors], "o-", label="noisy input")
plt.plot(levels, [e[1] for e in errors], "s-", label="denoised output")
plt.axvline(noise_factor, color="gray", linestyle="--", label="training noise level")
plt.xlabel("noise_factor at test time")
plt.ylabel("MSE with the clean images")
plt.title("Robustness to the noise level")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

for f, (b, a) in zip(levels, errors):
    print(f"noise_factor = {f:.1f} | noisy: {b:.5f} | denoised: {a:.5f}")
```

The model was trained with `noise_factor = 0.5`. This block tests it with less noise and with more noise.

Question 3.1. Why is it impossible for this network to simply *copy* its input?

Question 3.2. Look at the curve of 3.9 at `noise_factor = 0`. The model makes clean images slightly **worse**. Why?

Question 3.3. For which noise levels is the model useful? What happens far from the training level?

Question 3.4. (Exercise) In `add_noise`, replace the Gaussian noise with *salt-and-pepper* noise (20% of the pixels set to 0 or 1 at random). Train again and show the results.

Section 4 — A variational autoencoder (VAE)

Goal. Build a model that can **generate** new digits.

$$\text{image } (784) \to \text{hidden } (400) \to \mu,\ \log\sigma^2\ (20) \to z\ (20) \to \text{hidden } (400) \to \text{reconstruction } (784)$$

4.2 Load the data

```
transform = transforms.ToTensor()
train_data = datasets.MNIST(root="data", train=True, download=True, transform=transform)
test_data = datasets.MNIST(root="data", train=False, download=True, transform=transform)

batch_size = 128
train_loader = torch.utils.data.DataLoader(train_data, batch_size=batch_size, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_data, batch_size=batch_size, shuffle=False)

images, labels = next(iter(test_loader))
print("Training images:", len(train_data), "| Test images:", len(test_data))
print("One batch of images:", tuple(images.shape))
```

Same data as before, with a larger batch (128).

4.3 The model

```
class VAE(nn.Module):
    def __init__(self, input_dim=784, hidden_dim=400, latent_dim=20):
        super().__init__()
        self.latent_dim = latent_dim
        # ---------- encoder ----------
        self.linear_encode = nn.Linear(input_dim, hidden_dim)
        self.linear_mu = nn.Linear(hidden_dim, latent_dim)        # head 1: the mean
        self.linear_logvar = nn.Linear(hidden_dim, latent_dim)    # head 2: log(sigma^2)
        # ---------- decoder ----------
        self.linear_decode = nn.Linear(latent_dim, hidden_dim)
        self.linear_out = nn.Linear(hidden_dim, input_dim)

    def encode(self, x):
        h = F.relu(self.linear_encode(x))
        return self.linear_mu(h), self.linear_logvar(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)      # sigma = exp(logvar / 2)
        eps = torch.randn_like(std)        # noise from N(0, 1), drawn OUTSIDE the network
        return mu + eps * std              # z = mu + sigma * eps

    def decode(self, z):
        h = F.relu(self.linear_decode(z))
        return torch.sigmoid(self.linear_out(h))    # pixel values in [0, 1]

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar

model = VAE(latent_dim=20).to(device)
print(model)
print("Number of parameters:", sum(p.numel() for p in model.parameters()))
```

- The encoder has **two heads**: `linear_mu` gives the mean μ, `linear_logvar` gives $\log \sigma^2$. Each image becomes a small Gaussian cloud $\mathcal{N}(\mu, \sigma^2)$, not a single point.
- Why `logvar` and not σ? A linear layer can output any number, but a variance must be positive. With `logvar`, $\sigma = \exp(\text{logvar}/2)$ is always positive.
- `reparameterize` is the **reparameterization trick** of the lecture: the noise `eps` is drawn outside the network, then $z = \mu + \sigma \cdot \epsilon$. The gradient can go back to μ and σ.
- `forward` returns three things: the reconstruction, `mu` and `logvar`. The loss needs all of them.

4.4 The loss

```python
def loss_function(x_hat, x, mu, logvar, beta=1.0):
    # reconstruction term: binary cross-entropy, summed over the pixels and the batch
    rec = F.binary_cross_entropy(x_hat, x, reduction="sum")
    # KL term: closed form between N(mu, sigma^2) and N(0, 1)
    kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return rec + beta * kl, rec, kl

# quick check with the untrained model
x = images.to(device).view(-1, 784)
x_hat, mu, logvar = model(x)
loss, rec, kl = loss_function(x_hat, x, mu, logvar)
print(f"Per image, before training: reconstruction = {rec.item() / x.size(0):.1f}"
      f" | KL = {kl.item() / x.size(0):.2f}")
```

This is the formula of the lecture:

$$\mathcal{L} = \underbrace{\mathrm{BCE}(x, \hat{x})}_{\text{reconstruction}} + \beta \cdot \underbrace{\Big(-\tfrac{1}{2} \sum_j \big(1 + \mathrm{logvar}_j - \mu_j^2 - e^{\mathrm{logvar}_j}\big)\Big)}_{\text{KL divergence to } \mathcal{N}(0,I)}$$

- `reduction="sum"`: the BCE is summed over the 784 pixels and over the batch. Both terms must use the same convention (sum), otherwise their balance is wrong.
- The function also returns the two terms separately, so that we can plot them.

4.5 Training

```python
def run_epoch(model, loader, optimizer=None):
    # One pass over the data. With an optimizer: training. Without: evaluation only.
    training = optimizer is not None
    if training:
        model.train()
    else:
        model.eval()
    total_rec, total_kl = 0.0, 0.0
    with torch.set_grad_enabled(training):
        for images, _ in loader:
            x = images.to(device).view(-1, 784)
            x_hat, mu, logvar = model(x)
            loss, rec, kl = loss_function(x_hat, x, mu, logvar)
            if training:
                optimizer.zero_grad()
                loss.backward()
                optimizer.step()
            total_rec += rec.item()
            total_kl += kl.item()
    n = len(loader.dataset)
    return total_rec / n, total_kl / n       # averages per image

def train_vae(model, n_epochs, lr=0.001):
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    history = {"train_rec": [], "train_kl": [], "test_rec": [], "test_kl": []}
    for epoch in range(1, n_epochs + 1):
        train_rec, train_kl = run_epoch(model, train_loader, optimizer)
        test_rec, test_kl = run_epoch(model, test_loader)
        for key, value in zip(history, [train_rec, train_kl, test_rec, test_kl]):
            history[key].append(value)
        print(f"Epoch {epoch:2d}/{n_epochs} | train: rec {train_rec:6.2f} + KL {train_kl:5.2f}"
              f" = {train_rec + train_kl:6.2f} | test: {test_rec + test_kl:6.2f}")
    return history
```

```
n_epochs = 15
history = train_vae(model, n_epochs)
```

- `run_epoch` does one pass over the data. With an optimizer it trains; without, it only evaluates. `torch.set_grad_enabled(training)` turns the gradients on or off.
- It returns the reconstruction term and the KL term as averages **per image**.

Expected output (first and last lines):

```
Epoch  1/15 | train: rec 151.02 + KL 15.11 = 166.13 | test: 127.82
Epoch  2/15 | train: rec  99.12 + KL 22.35 = 121.47 | test: 115.70
...
Epoch 14/15 | train: rec  79.99 + KL 25.26 = 105.25 | test: 104.86
Epoch 15/15 | train: rec  79.71 + KL 25.27 = 104.98 | test: 104.85
```

4.6 The two terms of the loss

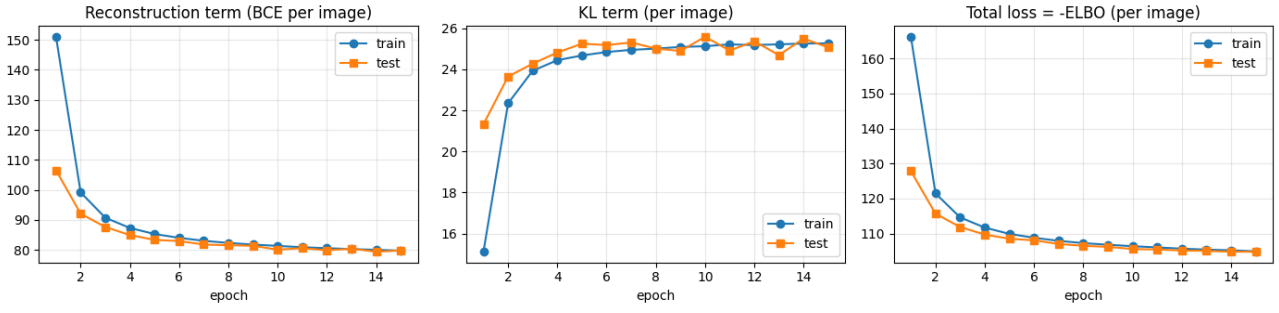
```
epochs = range(1, n_epochs + 1)
fig, axes = plt.subplots(1, 3, figsize=(14, 3.5))

axes[0].plot(epochs, history["train_rec"], "o-", label="train")
axes[0].plot(epochs, history["test_rec"], "s-", label="test")
axes[0].set_title("Reconstruction term (BCE per image)")

axes[1].plot(epochs, history["train_kl"], "o-", label="train")
axes[1].plot(epochs, history["test_kl"], "s-", label="test")
axes[1].set_title("KL term (per image)")

axes[2].plot(epochs, np.add(history["train_rec"], history["train_kl"]), "o-", label="train")
axes[2].plot(epochs, np.add(history["test_rec"], history["test_kl"]), "s-", label="test")
axes[2].set_title("Total loss = -ELBO (per image)")

for ax in axes:
    ax.set_xlabel("epoch")
    ax.legend()
    ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



The reconstruction term goes down. The KL term goes **up** at the start: to reconstruct well, the encoder moves the codes away from the prior $\mathcal{N}(0, I)$, and it pays the KL "tax" for it.

4.7 and 4.8 Reconstructions and generation of new digits

```
model.eval()
images, labels = next(iter(test_loader))
```

```
with torch.no_grad():
    x_hat, mu, logvar = model(images.to(device).view(-1, 784))

show_rows([images, x_hat], ["Original test images", "VAE reconstructions"])
```

```
# GENERATION: we do not use the encoder. We sample z from the prior N(0, I) and decode.
torch.manual_seed(0)
with torch.no_grad():
    z = torch.randn(30, model.latent_dim, device=device)
    generated = model.decode(z)

show_rows([generated[:10], generated[10:20], generated[20:30]],
          ["New digits generated from random z", "", ""])
```



To generate, we do **not** use the encoder: we sample $z \sim \mathcal{N}(0, I)$ and we decode. Compare with 1.10: random codes now give digits, because the KL term has organized the latent space around $\mathcal{N}(0, I)$.

4.9 Interpolation in the latent space

```
def interpolate(model, image_a, image_b, n=10):
    # walk in a straight line, in the latent space, from the code of a to the code of b
    with torch.no_grad():
        mu_a, _ = model.encode(image_a.to(device).view(1, 784))
        mu_b, _ = model.encode(image_b.to(device).view(1, 784))
        t = torch.linspace(0, 1, n, device=device).view(n, 1)
        z = (1 - t) * mu_a + t * mu_b        # n points between mu_a and mu_b
        return model.decode(z)

# take the first "1" and the first "7" of the test batch
idx_a = (labels == 1).nonzero()[0].item()
idx_b = (labels == 7).nonzero()[0].item()
show_rows([interpolate(model, images[idx_a], images[idx_b])],
          ["Interpolation in the latent space: from a 1 to a 7"])
```



We move in a straight line from the code of a “1” to the code of a “7” and we decode each point. The digit changes smoothly: the latent space has no holes.

4.10 A VAE with a 2-D latent space

```
torch.manual_seed(0)
model_2d = VAE(latent_dim=2).to(device)
n_epochs_2d = 10
history_2d = train_vae(model_2d, n_epochs_2d)
```

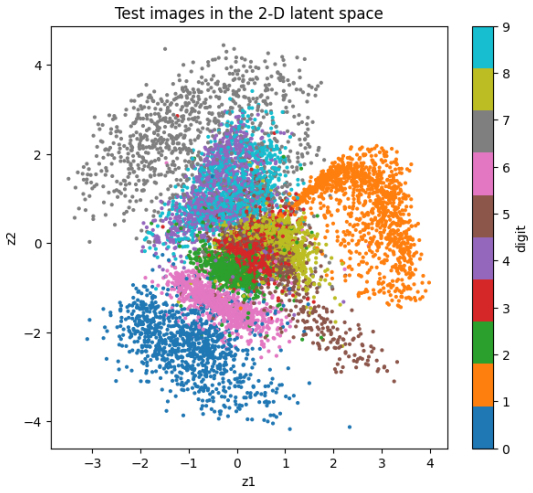
```
# Encode the whole test set and keep the means mu (2 numbers per image)
model_2d.eval()
all_mu, all_labels = [], []
with torch.no_grad():
    for images, labels in test_loader:
        mu, _ = model_2d.encode(images.to(device).view(-1, 784))
        all_mu.append(mu.cpu())
        all_labels.append(labels)
all_mu = torch.cat(all_mu).numpy()
all_labels = torch.cat(all_labels).numpy()

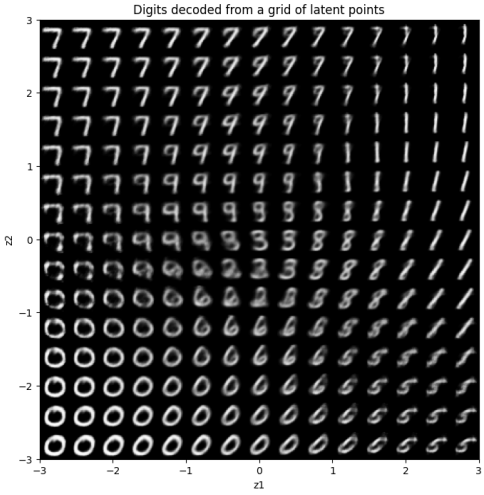
plt.figure(figsize=(7, 6))
points = plt.scatter(all_mu[:, 0], all_mu[:, 1], c=all_labels, cmap="tab10", s=4)
plt.colorbar(points, ticks=range(10), label="digit")
plt.xlabel("z1")
plt.ylabel("z2")
plt.title("Test images in the 2-D latent space")
plt.show()
```

```
# Decode a regular grid of points z = (z1, z2)
n = 15
grid = torch.linspace(-3, 3, n)
canvas = np.zeros((n * 28, n * 28))
with torch.no_grad():
    for i, z2 in enumerate(torch.flip(grid, [0])):          # top row = largest z2
        for j, z1 in enumerate(grid):
            z = torch.tensor([[z1.item(), z2.item()]], device=device)
            canvas[i * 28:(i + 1) * 28, j * 28:(j + 1) * 28] = \
                model_2d.decode(z).view(28, 28).cpu().numpy()

plt.figure(figsize=(8, 8))
plt.imshow(canvas, cmap="gray", extent=[-3, 3, -3, 3])
plt.xlabel("z1")
plt.ylabel("z2")
plt.title("Digits decoded from a grid of latent points")
plt.show()
```

With only 2 latent dimensions the images are less sharp, but we can **draw** the latent space. Left: each test image is a point (its mean μ), coloured by its label; the labels were not used for training. Right: the digits decoded from a regular grid of points z.





Question 4.1. In 4.6, why does the KL term increase during the first epochs?

Question 4.2. Compare the reconstructions of the VAE with those of Sections 1 and 2. Which ones are sharper? Why?

Question 4.3. In the 2-D latent space, which digits are neighbours? Which ones overlap? Is it surprising?

Question 4.4. (Exercise) Train the VAE with `beta = 0` and with `beta = 5` (add a `beta` argument to `run_epoch` and `train_vae`). For each value, show reconstructions and generated digits, and explain what you see.

Question 4.5. (Exercise) Replace the linear layers with the convolutional encoder and decoder of the lecture (`Conv2d` with `stride=2`, `ConvTranspose2d`).

Summary

Model	Input / target	Code	Parameters	Loss
Linear AE	image / same image	32 values	50 992	MSE
Convolutional AE	image / same image	$4 \times 7 \times 7$	1 077	MSE
Denoising AE	noisy image / clean image	$8 \times 3 \times 3$	9 585	MSE
VAE	image / same image	μ and $\log \sigma^2$ (20 each)	652 824	BCE + KL

Results of our reference run (CPU, default number of epochs):

Model	Epochs	Result on the test set
Linear AE (code of 32)	10	MSE = 0.0141
Convolutional AE, transposed convolutions	10	MSE = 0.0088
Convolutional AE, upsampling + convolution	10	MSE = 0.0060
Denoising AE (noise factor 0.5)	5	MSE with the clean images: 0.1157 (noisy input) → 0.0281 (output)
VAE, 20 latent dimensions	15	loss per image (reconstruction + KL) = 104.8
VAE, 2 latent dimensions	10	loss per image (reconstruction + KL) = 153.5

Your values can be a little different (other hardware, other versions), but they should be close.